Кочарян Элина Самвеловна
БПИ-244
АКОС идз-3

Вариант-16 (на 10 баллов)

Задача о социалистической гостинице. В гостинице 10 одноместных номеров. Клиенты гостиницы снимают номер на одни или несколько суток (задается при создании клиента). Если в гостинице нет свободных номеров, клиенты не уходят, а устраиваются на рядом с гостиницей на скамейках и ждут в порядке очереди, пока любой из номеров не освободится (других гостиниц в городе нет). Создать многопроцессное приложение, моделирующее работу гостиницы.Каждого клиента и гостиницу (точнее ее администратора) моделировать отдельным процессом.

П1

Администратор (родитель): управляет заселением/выселением, ведёт очередь ожидания, обрабатывает события.
Клиенты (N дочерних процессов): формируют заявки на заселение и затем выезжают.

SharedState в POSIX SHM (shm_open - ftruncate - mmap), содержит:

- occupied[10] - состояние номеров (PID или -1 свободно);
- Очередь событий ev[] и семафоры ev_mutex, ev_items, ev_slots;
- Массив заявок req[] и массив семафоров ответов cli_sem[] (по одному на клиента);
- Флаг shutting_down для режима завершения.

События (клиент → администратор):

EV_REQUEST(cli_index, pid) -заявка на заселение;
EV_CHECKOUT(cli_index, room, pid) -освобождение номера.

Администратор в цикле:

1. Обрабатывает входящие события (назначение номера либо постановка в FIFO-очередь ожидания);
2. Обрабатывает выселения (освобождение номера и, если есть, заселение следующего из очереди);
3. Периодически убирает завершившихся детей waitpid.

Для наглядности каждый процесс пишется со временем
[HH:MM:SS.mmm | +ms][РОЛЬ][pid=...]

Семафоры: неименованные POSIX sem_t с pshared=1, размещены в разделяемой памяти:
ev_items/ev_slots - счётчики занятых/свободных слотов очереди событий
ev_mutex - мьютекс очереди событий
cli_sem[i] - семафор для ответа клиенту

Очередь событий:
ev_push: sem_wait(ev_slots) - sem_wait(ev_mutex) - запись - sem_post(ev_mutex) - sem_post(ev_items);
ev_pop_timed: реализован через sem_trywait(ev_items) + короткий nanosleep(1ms) до истечения таймаута.

Обработка CTRL + C

Администратор перехватывает SIGINT (Ctrl+C), ставит флаг shutting_down=1, клиентские процессы сигнал игнорируют. Все ожидающие в очереди отменяются: админ ставит им assigned = -1 и они выходят. Уже заселённые клиенты доживают свой срок, делают checkout и завершаются - новых заселений после SIGINT нет.
Когда все дочерние процессы завершились, админ печатает сводку, очищает ресурсы и выходит.

Пример:

```
^C[17:40:40.712 | +00000409 ms][ADMIN][pid=24279] SIGINT -> shutdown: cancel waiting (2)
[17:40:41.542 | +00001239 ms][ADMIN][pid=24279] shutdown summary: checkouts=10, finished_after_sigint=12, took ~830 ms
[17:40:41.543 | +00001240 ms][ADMIN][pid=24279] all clients finished, admin exits
[17:40:41.543 | +00001240 ms][ADMIN][pid=24279] cleanup done, exit
```

Соответствие требованиям 4–6 баллов

1. Сценарий и отображение сущностей- один админ (родитель) + N клиентов (дочерние)
2. Взаимодействие с использованием неименованных POSIX семафоров - да (sem_t в SHM).
3. Обмен данными через POSIX shared memory - да (shm_open/mmap).
4. Корректное завершение и по сигналу - есть обработка Ctrl+ C
5. Понятный вывод ролей и действий каждого процесса - да (ADMIN/CLIENT, pid, комнаты)
6. Удаление семафоров и разделяемой памяти - полная очистка. В конце main(): sem_destroy(...) для всех, munmap(S, ...), shm_unlink(SHM_NAME).

Запуск:

gcc -O2 -Wall -Wextra -pthread main.c -o p1_hotel
./p1_hotel --clients 3 --min-d 1 --max-d 3 --day-ms 400

Пример:

./p1_hotel --clients 4 --min-d 1 --max-d 3 --day-ms 500

```
elina@WIN-7S7OPA43VDQ:~/new_folder/new_folder/.vscode/AKOC9/AKOC-idz3/p1$ ./p1_hotel --clients 4 --min-d 1 --max-d 3
[21:34:05.408 | +00000000 ms][ADMIN][pid=28985] build: Nov 29 2025 17:37:02
[21:34:05.409 | +00000001 ms][ADMIN][pid=28985] start: clients=4, nights=[1..3], day_ms=500
[21:34:05.410 | +00000002 ms][CLIENT][pid=28986] request: need 2 night(s)
[21:34:05.410 | +00000002 ms][CLIENT][pid=28987] request: need 1 night(s)
[21:34:05.411 | +00000003 ms][CLIENT][pid=28988] request: need 3 night(s)
[21:34:05.411 | +00000003 ms][ADMIN][pid=28985] check-in pid=28986 to room #0 (free immediately)
[21:34:05.412 | +00000004 ms][CLIENT][pid=28986] got room #0, staying 2 night(s)
[21:34:05.412 | +00000004 ms][ADMIN][pid=28985] check-in pid=28987 to room #1 (free immediately)
[21:34:05.412 | +00000004 ms][CLIENT][pid=28987] got room #1, staying 1 night(s)
[21:34:05.412 | +00000004 ms][ADMIN][pid=28985] check-in pid=28988 to room #2 (free immediately)
[21:34:05.412 | +00000004 ms][CLIENT][pid=28988] got room #2, staying 3 night(s)
[21:34:05.412 | +00000004 ms][ADMIN][pid=28985] check-in pid=28989 to room #3 (free immediately)
[21:34:05.412 | +00000004 ms][CLIENT][pid=28989] request: need 1 night(s)
[21:34:05.412 | +00000004 ms][CLIENT][pid=28989] got room #3, staying 1 night(s)
[21:34:05.912 | +00000504 ms][CLIENT][pid=28989] checkout room #3
[21:34:05.912 | +00000504 ms][CLIENT][pid=28987] checkout room #1
[21:34:05.913 | +00000505 ms][ADMIN][pid=28985] room #1 is free (pid=28987 checked out)
[21:34:05.913 | +00000505 ms][ADMIN][pid=28985] client pid=28987 finished (1/4)
[21:34:05.913 | +00000505 ms][ADMIN][pid=28985] client pid=28989 finished (2/4)
[21:34:05.913 | +00000505 ms][ADMIN][pid=28985] room #3 is free (pid=28989 checked out)
[21:34:06.412 | +00001004 ms][CLIENT][pid=28986] checkout room #0
[21:34:06.413 | +00001005 ms][ADMIN][pid=28985] room #0 is free (pid=28986 checked out)
[21:34:06.413 | +00001005 ms][ADMIN][pid=28985] client pid=28986 finished (3/4)
[21:34:06.912 | +00001504 ms][CLIENT][pid=28988] checkout room #2
[21:34:06.912 | +00001504 ms][ADMIN][pid=28985] room #2 is free (pid=28988 checked out)
[21:34:07.113 | +00001705 ms][ADMIN][pid=28985] client pid=28988 finished (4/4)
[21:34:07.113 | +00001705 ms][ADMIN][pid=28985] all clients finished, admin exits
[21:34:07.114 | +00001706 ms][ADMIN][pid=28985] cleanup done, exit
```

Пример с очередью(11 чел):

./p1_hotel --clients 11 --min-d 1 --max-d 3 --day-ms 500

```
elina@WIN-7S7OPA43VDQ:~/new_folder/new_folder/.vscode/AKOC9/AKOC-idz3/p1$ ./p1_hotel --clients 11 --min-d 1 --max-d 3 --day-ms 500
[21:01:30.895 | +00000000 ms][ADMIN][pid=20396] start: clients=11, nights=[1..3], day_ms=500
[21:01:30.895 | +00000000 ms][CLIENT][pid=20397] request: need 1 night(s)
[21:01:30.895 | +00000000 ms][CLIENT][pid=20398] request: need 1 night(s)
[21:01:30.896 | +00000001 ms][CLIENT][pid=20399] request: need 1 night(s)
[21:01:30.896 | +00000001 ms][CLIENT][pid=20400] request: need 1 night(s)
[21:01:30.896 | +00000001 ms][CLIENT][pid=20401] request: need 1 night(s)
[21:01:30.896 | +00000001 ms][CLIENT][pid=20402] request: need 1 night(s)
[21:01:30.897 | +00000002 ms][CLIENT][pid=20403] request: need 1 night(s)
[21:01:30.897 | +00000002 ms][CLIENT][pid=20404] request: need 1 night(s)
[21:01:30.897 | +00000002 ms][CLIENT][pid=20405] request: need 1 night(s)
[21:01:30.897 | +00000002 ms][CLIENT][pid=20406] request: need 2 night(s)
[21:01:30.897 | +00000002 ms][ADMIN][pid=20396] check-in pid=20397 to room #0 (free immediately)
[21:01:30.897 | +00000002 ms][CLIENT][pid=20397] got room #0, staying 1 night(s)
[21:01:30.897 | +00000002 ms][ADMIN][pid=20396] check-in pid=20398 to room #1 (free immediately)
[21:01:30.897 | +00000002 ms][CLIENT][pid=20398] got room #1, staying 1 night(s)
[21:01:30.897 | +00000002 ms][CLIENT][pid=20407] request: need 3 night(s)
[21:01:30.897 | +00000002 ms][ADMIN][pid=20396] check-in pid=20400 to room #2 (free immediately)
[21:01:30.898 | +00000003 ms][ADMIN][pid=20396] check-in pid=20399 to room #3 (free immediately)
[21:01:30.898 | +00000003 ms][CLIENT][pid=20400] got room #2, staying 1 night(s)
[21:01:30.898 | +00000003 ms][CLIENT][pid=20399] got room #3, staying 1 night(s)
[21:01:30.898 | +00000003 ms][ADMIN][pid=20396] check-in pid=20401 to room #4 (free immediately)
[21:01:30.898 | +00000003 ms][CLIENT][pid=20401] got room #4, staying 1 night(s)
[21:01:30.898 | +00000003 ms][ADMIN][pid=20396] check-in pid=20402 to room #5 (free immediately)
[21:01:30.898 | +00000003 ms][ADMIN][pid=20396] check-in pid=20403 to room #6 (free immediately)
[21:01:30.898 | +00000003 ms][CLIENT][pid=20402] got room #5, staying 1 night(s)
[21:01:30.898 | +00000003 ms][CLIENT][pid=20403] got room #6, staying 1 night(s)
[21:01:30.898 | +00000003 ms][ADMIN][pid=20396] check-in pid=20404 to room #7 (free immediately)
[21:01:30.898 | +00000003 ms][CLIENT][pid=20404] got room #7, staying 1 night(s)
[21:01:30.898 | +00000003 ms][ADMIN][pid=20396] check-in pid=20405 to room #8 (free immediately)
[21:01:30.898 | +00000003 ms][CLIENT][pid=20405] got room #8, staying 1 night(s)
[21:01:30.898 | +00000003 ms][ADMIN][pid=20396] check-in pid=20406 to room #9 (free immediately)
[21:01:30.898 | +00000003 ms][CLIENT][pid=20406] got room #9, staying 2 night(s)
[21:01:30.898 | +00000003 ms][ADMIN][pid=20396] no vacancy -> queued (len=1)
[21:01:31.398 | +00000503 ms][CLIENT][pid=20398] checkout room #1
[21:01:31.398 | +00000503 ms][CLIENT][pid=20397] checkout room #0
[21:01:31.399 | +00000504 ms][CLIENT][pid=20402] checkout room #5
[21:01:31.399 | +00000504 ms][CLIENT][pid=20399] checkout room #3
[21:01:31.399 | +00000504 ms][CLIENT][pid=20403] checkout room #6
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] room #1 is free (pid=20398 checked out)
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] from queue -> check-in pid=20407 to #1 (queue left=0)
[21:01:31.399 | +00000504 ms][CLIENT][pid=20401] checkout room #4
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] client pid=20397 finished (1/11)
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] client pid=20398 finished (2/11)
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] client pid=20399 finished (3/11)
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] client pid=20402 finished (4/11)
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] client pid=20403 finished (5/11)
[21:01:31.399 | +00000504 ms][CLIENT][pid=20400] checkout room #2
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] room #0 is free (pid=20397 checked out)
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] room #5 is free (pid=20402 checked out)
[21:01:31.400 | +00000505 ms][CLIENT][pid=20405] checkout room #8
[21:01:31.399 | +00000504 ms][ADMIN][pid=20396] client pid=20401 finished (6/11)
[21:01:31.400 | +00000505 ms][ADMIN][pid=20396] room #3 is free (pid=20399 checked out)
[21:01:31.400 | +00000505 ms][CLIENT][pid=20404] checkout room #7
[21:01:31.400 | +00000505 ms][ADMIN][pid=20396] room #6 is free (pid=20403 checked out)
[21:01:31.400 | +00000505 ms][ADMIN][pid=20396] room #4 is free (pid=20401 checked out)
[21:01:31.400 | +00000505 ms][ADMIN][pid=20396] room #2 is free (pid=20400 checked out)
[21:01:31.400 | +00000505 ms][ADMIN][pid=20396] room #8 is free (pid=20405 checked out)
[21:01:31.400 | +00000505 ms][ADMIN][pid=20396] room #7 is free (pid=20404 checked out)
[21:01:31.400 | +00000505 ms][CLIENT][pid=20407] got room #1, staying 3 night(s)
[21:01:31.600 | +00000705 ms][ADMIN][pid=20396] client pid=20400 finished (7/11)
[21:01:31.600 | +00000705 ms][ADMIN][pid=20396] client pid=20404 finished (8/11)
[21:01:31.600 | +00000705 ms][ADMIN][pid=20396] client pid=20405 finished (9/11)
[21:01:31.898 | +00001003 ms][CLIENT][pid=20406] checkout room #9
[21:01:31.898 | +00001003 ms][ADMIN][pid=20396] room #9 is free (pid=20406 checked out)
[21:01:31.898 | +00001003 ms][ADMIN][pid=20396] client pid=20406 finished (10/11)
[21:01:32.900 | +00002005 ms][CLIENT][pid=20407] checkout room #1
[21:01:32.900 | +00002005 ms][ADMIN][pid=20396] room #1 is free (pid=20407 checked out)
[21:01:33.101 | +00002206 ms][ADMIN][pid=20396] client pid=20407 finished (11/11)
[21:01:33.101 | +00002206 ms][ADMIN][pid=20396] all clients finished, admin exits
[21:01:33.101 | +00002206 ms][ADMIN][pid=20396] cleanup done, exit
```

П2

Стало две независимые программы - p2_admin и p2_client, запускаются в разных консолях**:**

p2_admin: создаёт/инициализирует общий сегмент SHM и именованные семафоры, обрабатывает события, ведёт очередь ожидания, заселяет/выселяет.

p2_client: подключается к уже запущенному админу, размещает заявку, ожидает назначение, живёт нужное число суток, затем освобождает номер.

Что изменилось:

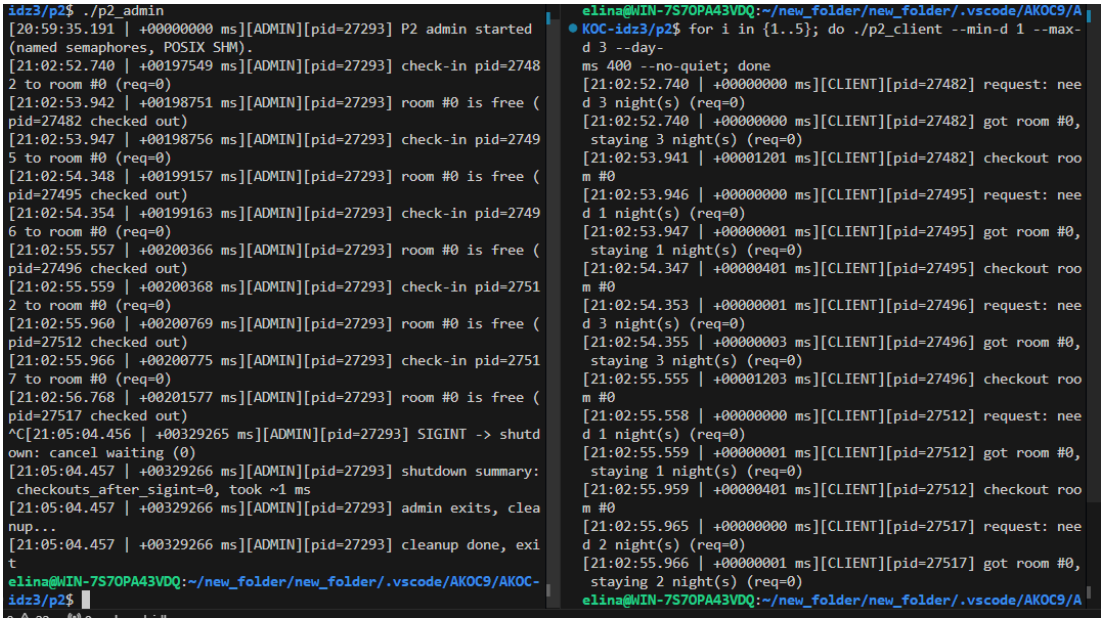
- Добавились именованные семафоры через sem_open/sem_unlink: /p2_ev_mutex, /p2_ev_items, /p2_ev_slots, /p2_req_mutex + персональный семафор клиента /p2_cli_<pid>.
- Появился сегмент /p2_hotel_shm с таблицей req[REQ_CAP], где у каждой заявки хранится ещё и sem_name (имя персонального семафора клиента). Очередь событий сохранилась, но поле события теперь ссылается на req_index вместо cli_index.
- Админ создаёт SHM и именованные семафоры- клиент пытается подключиться (повтор с небольшим сном), корректно сообщает, если админ не запущен.

Соответствие требованиям 7–8 баллов:

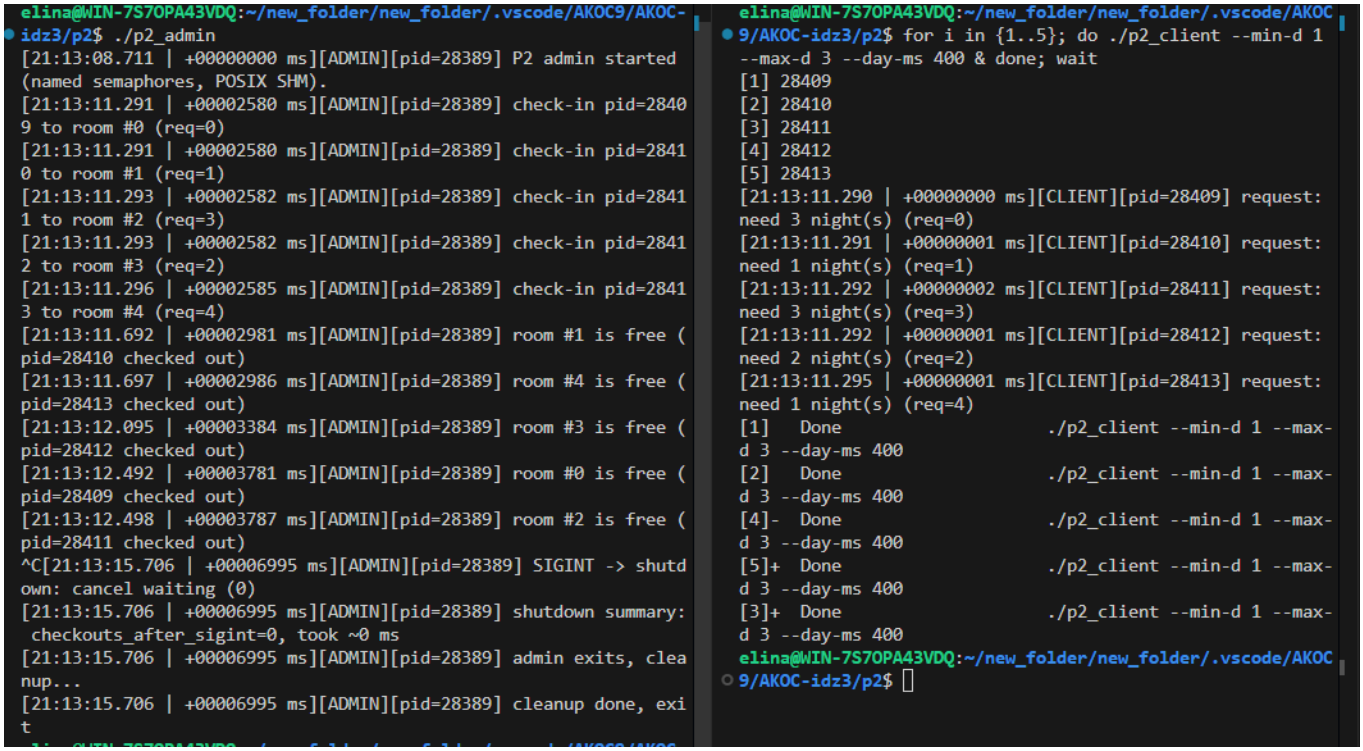
1. Именованные POSIX-семафоры - sem_open/sem_unlink: /p2_ev_mutex, /p2_ev_items, /p2_ev_slots, /p2_req_mutex, а также персональные семафоры клиента /p2_cli_<pid>
2. POSIX shared memory - общий сегмент /p2_hotel_shm через shm_open - ftruncate - mmap. В нём SharedState: занятость комнат и тд…
3. Завершение как в П1 -админ ловит SIGINT (sigaction, SA_RESTART), ставит shutting_down=1, отменяет ожидающих (устанавливает assigned=-1 и делает sem_post на /p2_cli_<pid>). Заселённые доживают и делают чекаут. Очистка: у админа sem_close/sem_unlink всех именованных семафоров и shm_unlink; у клиента закрытие и sem_unlink персонального семафора
4. Логи в своей консоли -отдельные процессы, которые пишут каждый свой поток

Пример последовательный вывод (по одному клиенту)

for i in {1..5}; do ./p2_client --min-d 1 --max-d 3 --day-ms 400 --no-quiet; done



Пример параллельный вывод:
for i in {1..5}; do ./p2_client --min-d 1 --max-d 3 --day-ms 400 & done; wait



Запуск:
gcc -O2 -Wall -Wextra -pthread p2_admin.c -o p2_admin
gcc -O2 -Wall -Wextra -pthread p2_client.c -o p2_client

консоль 1
./p2_admin

консоль 2
./p2_client --min-d 1 --max-d 3 --day-ms 400
или
for i in {1..3}; do ./p2_client --min-d 1 --max-d 3 --day-ms 400 & done; wait

ПЗ

p3_observer

1. Создаёт именованный канал /tmp/p3_observer.fifo
2. Читает строки в неблокирующем режиме, аккуратно собирает сообщения, печатает единый сводный лог.
3. Держит фиктивный дескриптор записи, чтобы избежать постоянного EOF, корректно завершает работу по Ctrl+C и удаляет FIFO (unlink).

В **p3_admin** и **p3_client** добавлен вывод ключевых сообщений в FIFO:
Отправка строк ≤ PIPE_BUF с обработкой EINTR/EAGAIN и короткими повторами.
Если наблюдатель не запущен (ошибка open/ENXIO) - запись тихо пропускается, локальный вывод в собственную консоль остаётся без изменений.*/

Наблюдатель -самостоятельный процесс. Не требуется для работы гостиницы, но при наличии получает полную картину событий (заявки, заселения, чекауты, сводки по Ctrl+C). Администратор и клиенты продолжают работать автономно, их логи дублируются в FIFO.

Как работает:

- Клиент размещает заявку (EV_REQUEST), ждёт персональный семафор.
- Админ заселяет (или ставит в FIFO-очередь ожидания), реагирует на чекауты (EV_CHECKOUT).
- Ключевые события печатаются в свои консоли и дублируются в FIFO наблюдателя.

Соответствие требованиям 9 баллов:

1. Те же подходы для завершения **-**корректная обработка `Ctrl+C`, отмена ожиданий, доживание заселённых, полная очистка ресурсов.
2. Независимый запуск всех процессов **-**наблюдатель, админ и клиенты стартуют отдельно; наблюдатель получает данные через именованный канал FIFO**;** админ/клиенты дополнительно передают свою информацию наблюдателю, сохраняя свой вывод в собственных консолях.

Запуск:

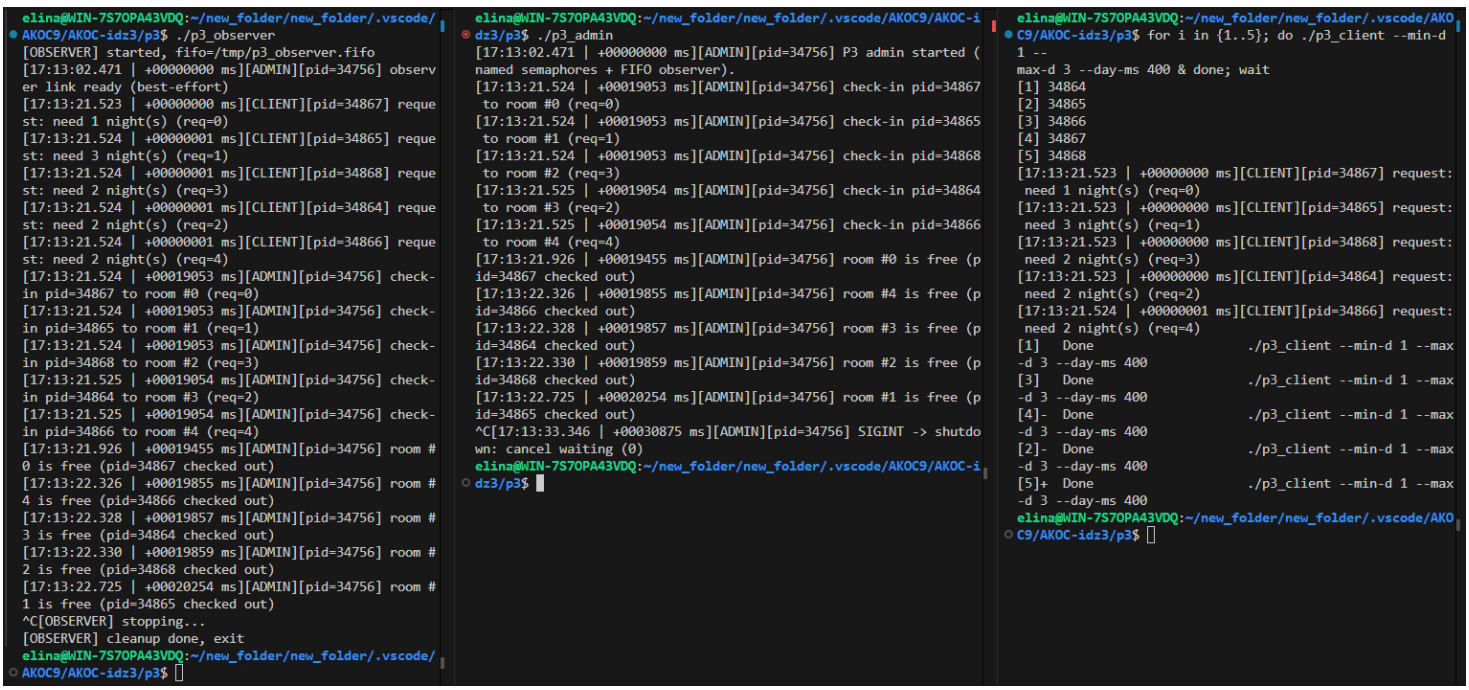
```
gcc -O2 -Wall -Wextra -pthread p3_admin.c  -o p3_admin
gcc -O2 -Wall -Wextra -pthread p3_client.c -o p3_client
gcc -O2 -Wall -Wextra -pthread p3_observer.c -o p3_observer

# консоль 1
./p3_observer

# консоль 2
./p3_admin

# консоль 3
for i in {1..5}; do ./p3_client --min-d 1 --max-d 3 --day-ms 400 & done; wait
```

Результат:



П4

Несколько наблюдателей и каждый ведет одинаковую ленту событий в своей консоли.

Что добавлено:

1. множественные наблюдатели через fifo

   Каждый наблюдатель создаёт собственный канал, регистрируется, отправив путь своего fifo в общую трубу, читает свой fifo неблокирующе, аккуратно собирая строки.

2. рассылка администратором в нескольких получателей

Админ поддерживает список подключённых fifo. При каждом лог-сообщении админ формирует строку и рассылает её всем наблюдателям. При получении регистрации админ добавляет/обновляет наблюдателя и логирует событие

3. устойчивость к порядку запуска

Наблюдатели периодически повторно шлют регистрацию (если админ стартовал позже). Админ держит дескриптор регистрационного fifo, чтобы у читателя не было постоянного EOF.

4. поведение после ctrl+c (как в п1–п3)

- админ ловит SIGINT, переводит систему в shutting_down, отменяет ожидающих, позволяет заселённым корректно выселиться, печатает shutdown summary, очищает семафоры и shm.
- наблюдатели ловят SIGINT и завершаются, закрывая дескрипторы (их fifo можно не удалять- подключение возможно повторно).
- клиенты при отмене завершаются, при заселении -доживают срок и делают checkout.

Каналы наблюдателей - REG_FIFO="/tmp/p4_reg_fifo", плюс персональные "/tmp/p4_obs_<pid>.fifo"

Потоки:

клиент - (shm+семафоры) - админ - (рассылка) - все наблюдатели. Одновременно каждый процесс ведёт собственную консоль.

Соответствие требованиям 10 баллов:

1. завершение как раньше: SIGINT у админа - shutting_down, отмена ожиданий, доживание заселённых, сводка, очистка shm/семафоров, наблюдатели и клиенты корректно завершаются.
2. несколько наблюдателей работают одинаково и согласованно- админ ведёт список fifo-наблюдателей и рассылает одно и то же сообщение

Запуск:

```
gcc -O2 -Wall -Wextra -pthread p4_observer.c -o p4_observer -lrt
gcc -O2 -Wall -Wextra -pthread p4_admin.c    -o p4_admin    -lrt
gcc -O2 -Wall -Wextra -pthread p4_client.c   -o p4_client
```

#в 1 окне
./p4_observer

#в 2 окне
./p4_observer

#в 3 окне
./p4_admin

#в 4 окне
for i in {1..5}; do ./p4_client --min-d 1 --max-d 3 --day-ms 400 & done; wait

Результат:

